МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Языки программирования

Повторение основ работы с классами

Руководитель _____ А.В. Проскурин
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

Проверка и получение навыков: использования языковых средств и способов динамического выделения памяти; организации форматного ввода и вывода; Получение практических навыков разработки и отладки программ, использующих объекты класса и файловый ввод/вывод. Изучение различных реализаций конструкторов и областей их применения, а также аспектов перегрузки операций и работы с дружественными функциями.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с общей постановкой задачи.

2. Ознакомится с вариантом задания – соответствует вашему номеру в списке группы (при нехватке заданий вариант задания вычисляется как номер_в_списке_группы - количество_заданий).

3. Разработать класс согласно варианту задания.

4. Написать и отладить программу на подготовленных наборах тестовых данных.

5. Подготовить отчет по лабораторной работе. Отчет должен включать в себя:

• титульный лист;

• цель лабораторной работы;

• постановку задачи;

• текст программы с комментариями;

• демонстрацию работы программы (Снимки экрана при выполнении действий программы с описанием).

• краткие ответы на контрольные вопросы;

• выводы по лабораторной работе.

6. Защитить лабораторную работу перед преподавателем.

ПОСТАНОВКА ЗАДАЧИ

Оценка «Удовлетворительно» (Базовый уровень)

• **Цель**: Проверить понимание основ класса: инкапсуляция, конструкторы, геттеры/сеттеры, простые методы.

• **Что должен сделать студент:**

1. Объявить класс с private полями.
2. Реализовать конструкторы: по умолчанию, с параметрами, копирования.
3. Реализовать методы для ввода/вывода данных, получающие потоки istream/ostream в качестве параметром.
4. Реализовать геттеры и сеттеры для 1-2 полей.
5. Реализовать одну простую выборку (например, пункт 'a' из варианта), работающую с одиночным объектом (не с массивом), просто чтобы продемонстрировать работу методов.
6. В main() продемонстрировать создание нескольких объектов разными конструкторами, вызов их методов ввода/вывода, геттеров/сеттеров и одной выборки.

Оценка «Хорошо» (Продвинутый уровень)

• **Цель**: Работа с динамической памятью, файловый ввод/вывод, перегрузка операторов для потоков.

• **Что должен сделать студент:**

1. **Доработать программу с оценки «Удовлетворительно».**
2. Реализовать перегрузку операторов << и >> для ввода/вывода в поток. (Это
заменяет или дополняет старые методы print()/scan()).
3. Реализовать **статический метод** readFromFile(std::istream&) или аналогичную функцию, которая:
   - Принимает поток (например, ifstream).
   - Считывает количество объектов.
   - Динамически выделяет память под массив объектов.
   - Последовательно считывает данные для каждого объекта из потока (используя перегруженный >>!).
   - Возвращает указатель на созданный массив.
4. Реализовать **статический метод** writeToFile(std::ostream&, [массив и его размер]) или функцию, которая выводит массив объектов в поток (используя перегруженный <<!).
5. Продемонстрировать это в main(): чтение массива из файла, вывод его на экран, запись в другой файл.

Оценка «Отлично» (Углубленный уровень)

• **Цель**: Применение полиморфизма (через перегрузку операторов), алгоритмов сортировки, создание законченного приложения с интерфейсом.

• **Что должен сделать студент:**

1. **Доработать программу с оценки «Хорошо».**

2. Перегрузить операторы сравнения (например, <, >, ==), необходимые для сортировки по указанному в варианте полю.
3. Реализовать функцию сортировки массива объектов (например, методом пузырька или qsort), использующую перегруженные операторы сравнения.
4. Реализовать **все** выборки из варианта, работающие уже с массивом объектов.
5. Реализовать текстовое меню с помощью switch-case в цикле. Меню должно позволять:

- Загружать данные из файла.
- Выводить данные на экран/в файл.
- Выполнять все варианты выборок.
- Сортировать массив.
- Добавлять новый объект в массив (опционально, сложная задача).
- Выходить из программы.

Вариант №5. Класс Movie (Фильм)

Поля: Название, Режиссер, Год выхода, Жанр, Рейтинг IMDb, Длительность (мин).

Выборки:

а) Список фильмов заданного режиссера.

б) Список фильмов указанного жанра с рейтингом выше заданного.

в) Список фильмов, вышедших между двумя указанными годами.

Сортировка: по рейтингу IMDb (по убыванию).

ХОД РАБОТЫ

Файл **movie.h**:

```
#ifndef MOVIE_H        You, 2 недели назад via  PR #2 • инитиал коммит
#define MOVIE_H

/**
 * Сообщения при ошибке программы
 */

#define ERROR_MESSAGE_EMPTY_LIST "ERROR: The list is empty."
#define ERROR_MESSAGE_NOT_A_NUM "ERROR: Inserted text isn't a number."
#define ERROR_MESSAGE_INVALID_MENU_CHOICE "ERROR: Invalid menu option."
#define ERROR_MESSAGE_FILE_OPENING_FAIL "ERROR: Failed to open file."
#define ERROR_MESSAGE_FILE_READING_FAIL "ERROR: Failed to read from file."
#define ERROR_MESSAGE_FILE_WRITING_FAIL "ERROR: Failed to write to file."


#include <iostream>
#include <vector>
using namespace std;

You, 4 дня назад | 1 author (You)
class Movie {
private:
    string name;
    string director;
    int year;
    string genre;
    float rating;
    int length;

public:

    /**
     * Конструкторы класса (по умолчанию; с аргументами; копирование; удаление)
     */

    Movie();
    Movie(string myName, string myDirector, int myYear, string myGenre, float myRating, int myLength);
    Movie(const Movie& other);
    ~Movie();

    friend istream& operator>>(istream& is, Movie& mov);
    friend ostream& operator<<(ostream& os, const Movie& mov);

    /**
     * Геттеры (используются в выборках)
     */

    string getName() const { return name; }
    string getDirector() const { return director; }
    int getYear() const { return year; }
    string getGenre() const { return genre; }
    float getRating() const { return rating; }
    int getLength() const { return length; }


    /**
     * Методы сравнения по рейтингу
     */

    bool operator<(const Movie& other) const { return this->rating < other.rating; }
    bool operator>(const Movie& other) const { return this->rating > other.rating; }
```

```
    /**
     * Методы сравнения по рейтингу
     */

    bool operator<(const Movie& other) const { return this->rating < other.rating; }
    bool operator>(const Movie& other) const { return this->rating > other.rating; }

    /**
     * Методы для работы с файлами
     */

    static vector<Movie> read_from_file(const string& filename);
    static void write_to_file(const string& filename, const vector<Movie>& movies);

    /**
     * Компаратор для qsort
     */

    static int compare_movies_by_rating(const void* a, const void* b);

    /**
     * Выборки
     */

    static void select_by_director(vector<Movie>& movies, int count);
    static void select_by_genre_and_rating(vector<Movie>& movies, int count);
    static void select_by_years(vector<Movie>& movies, int count);

    /**
     * Сортировка фильмов по рейтингу
     */

    static void sort_by_rating(vector<Movie>& movies);

    /**
     * Метод для добавления фильма в массив
     */

    static void add_movie(vector<Movie>& movies, int count);
};

#endif
```

Файл **movie.cpp**:

```
#include "movie.h"        You, 2 недели назад via  PR #2 • инитиал коммит …
#include <iostream>
#include <fstream>
#include <algorithm>
using namespace std;

Movie::Movie():
    name(""),
    director(""),
    year(0),
    genre(""),
    rating(0),
    length(0) {}

Movie::Movie(string myName, string myDirector, int myYear, string myGenre, float myRating, int myLength):
    name(myName),
    director(myDirector),
    year(myYear),
    genre(myGenre),
    rating(myRating),
    length(myLength) {}

Movie::Movie(const Movie& other):
    name(other.name),
    director(other.director),
    year(other.year),
    genre(other.genre),
    rating(other.rating),
    length(other.length) {}

Movie::~Movie() {}

/// @brief Ввод полей объекта ч/з поток. Проверки на соответствие потока нужны для правильной работы read_from_file
/// @param is Входящий поток
/// @param mov Изменяемый объект
/// @return Поля объекта в виде потока
istream& operator>>(istream& is, Movie& mov) {
    if (&is == &cin) {
        cout << "Enter name: ";
    }
    is >> mov.name;

    if (&is == &cin) {
        cout << "Enter director: ";
    }
    is >> mov.director;

    if (&is == &cin) {
        cout << "Enter year: ";
    }
    is >> mov.year;

    if (&is == &cin) {
        cout << "Enter genre: ";
    }
    is >> mov.genre;

    if (&is == &cin) {
        cout << "Enter rating: ";
    }
    is >> mov.rating;

    if (&is == &cin) {
        cout << "Enter length: ";
    }
    is >> mov.length;

    return is;
}
```

```cpp
/// @brief Вывод полей объекта ч/з поток
/// @param os Выходящий поток
/// @param mov Считываемый объект
/// @return Поля объекта в виде потока
ostream& operator<<(ostream& os, const Movie& mov) {
    os << "Name: \"" << mov.name << "\"" << endl
       << "Director: \"" << mov.director << "\"" << endl
       << "Year: \"" << mov.year << "\"" << endl
       << "Genre: \"" << mov.genre << "\"" << endl
       << "Rating: \"" << mov.rating << "\"" << endl
       << "Length: \"" << mov.length << "\"" << endl;
    return os;
}

/// @brief Чтение инфы из файла
/// @param filename Название файла для открытия
/// @return Список прочитанных объектов
vector<Movie> Movie::read_from_file(const string& filename) {
    ifstream file(filename);
    vector<Movie> movies;

    if (!file.is_open()) {
        cout << ERROR_MESSAGE_FILE_OPENING_FAIL << endl;
        return movies;
    }

    int count;
    file >> count;

    for (int i = 0; i < count; ++i) {
        Movie temp;
        file >> temp;
        if(file.fail()) {
            cout << ERROR_MESSAGE_FILE_READING_FAIL << endl;
            movies.clear();
            break;
        }
        movies.push_back(temp);
    }

    file.close();
    if (!movies.empty()) {
        cout << "Data loaded successfully. Number of movies: " << movies.size() << endl;
    }

    return movies;
}


/// @brief Запись инфы в файл
/// @param filename Название файла для открытия
/// @param movies Список объектов для записи
void Movie::write_to_file(const string& filename, const vector<Movie>& movies) {
    ofstream file(filename);

    if (!file.is_open()) {
        cout << ERROR_MESSAGE_FILE_WRITING_FAIL << endl;
        return;
    }

    for (const auto& mov : movies) {
        file << mov << "\n";
    }

    file.close();
    cout << "Data saved to " << filename << endl;
}
```

```cpp
/// @brief Компаратор для qsort
/// @param a Первый фильм для сравнения
/// @param b Второй фильм для сравнения
/// @return Результат сравнения как int
int Movie::compare_movies_by_rating(const void* a, const void* b) {
    const Movie* movieA = (const Movie*)a;
    const Movie* movieB = (const Movie*)b;

    if (movieA->getRating() > movieB->getRating()) return -1;  // в порядке убывания
    else if (movieA->getRating() < movieB->getRating()) return 1;
    else return 0;
}

/// @brief Выборка а) по режиссёру
/// @param movies Массив фильмов
/// @param count Количество фильмов в массиве
void Movie::select_by_director(vector<Movie>& movies, int count) {
    if (movies.empty()) {
        cout << ERROR_MESSAGE_EMPTY_LIST << endl;
        return;
    }

    string director;
    cout << "Enter director name: ";
    cin >> director;

    vector<Movie> selected;
    for (int i = 0; i < count; ++i) {
        if (movies[i].getDirector() == director) {
            selected.push_back(movies[i]);
        }
    }

    if (selected.empty()) {
        cout << "No movies found by this director." << endl;
    } else {
        cout << "\n=== MOVIES BY DIRECTOR \"" << director << "\" ===" << endl;
        for (const Movie& m : selected) {
            cout << m << endl;
        }
    }
}
```

```
/// @brief Выборка б) по жанру + рейтинг выше указанного
/// @param movies Массив фильмов
/// @param count Количество фильмов в массиве
void Movie::select_by_genre_and_rating(vector<Movie>& movies, int count) {
    if (movies.empty()) {
        cout << ERROR_MESSAGE_EMPTY_LIST << endl;
        return;
    }

    string genre;
    float minRating;
    cout << "Enter genre: ";
    cin >> genre;
    cout << "Enter minimum rating: ";
    cin >> minRating;

    vector<Movie> selected;
    for (int i = 0; i < count; ++i) {
        if (movies[i].getGenre() == genre && movies[i].getRating() > minRating) {
            selected.push_back(movies[i]);
        }
    }

    if (selected.empty()) {
        cout << "No movies found with this genre and rating." << endl;
    } else {
        cout << "\n=== MOVIES OF GENRE \"" << genre << "\" WITH RATING > " << minRating << " ===" << endl;
        for (const Movie& m : selected) {
            cout << m << endl;
        }
    }
}

/// @brief Выборка в) год выхода в указанном промежутке
/// @param movies Массив фильмов
/// @param count Количество фильмов в массиве
void Movie::select_by_years(vector<Movie>& movies, int count) {
    if (movies.empty()) {
        cout << ERROR_MESSAGE_EMPTY_LIST << endl;
        return;
    }

    int startYear, endYear;
    cout << "Enter start year: ";
    cin >> startYear;
    cout << "Enter end year: ";
    cin >> endYear;

    vector<Movie> selected;
    for (int i = 0; i < count; ++i) {
        if (movies[i].getYear() >= startYear && movies[i].getYear() <= endYear) {
            selected.push_back(movies[i]);
        }
    }

    if (selected.empty()) {
        cout << "No movies found in the specified year range." << endl;
    } else {
        cout << "\n=== MOVIES BETWEEN " << startYear << " AND " << endYear << " ===" << endl;
        for (const Movie& m : selected) {
            cout << m << endl;
        }
    }
}

/// @brief Сортирует массив фильмов по рейтингу (в порядке убывания)
/// @param movies Массив фильмов
void Movie::sort_by_rating(vector<Movie>& movies) {
    sort(movies.begin(), movies.end(), [](const Movie& a, const Movie& b) {
        return a > b;
    });
    cout << "Movies sorted by rating (descending).\n";
```

Файл **main.cpp**:

```
#include "movie.h"        You, 2 недели назад via  PR #2 • инитиал коммит
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// File names for writing to/reading from
const string input_file_name = "input.txt";
const string output_file_name = "output.txt";

int main() {

    /// Массив фильмов
    vector<Movie> movies;
    /// Переменная для цикла while
    bool running = true;

    while (running) {
        cout << "\n====================================" << endl;
        cout << "                MAIN MENU" << endl;
        cout << "====================================" << endl;
        cout << "1. Load data from file" << endl;
        cout << "2. Display data on screen" << endl;
        cout << "3. Save data to file" << endl;
        cout << "4. Select movies by director" << endl;
        cout << "5. Select movies by genre and rating" << endl;
        cout << "6. Select movies by years" << endl;
        cout << "7. Sort movies by rating (qsort)" << endl;
        cout << "8. Add new movie" << endl;
        cout << "9. Exit" << endl;
        cout << "\nChoose an action: ";

        int choice;
        cin >> choice;

        switch (choice) {
            /// Загрузка данных из файла
            case 1: {
                movies = Movie::read_from_file(input_file_name);
                break;
            }
            /// Отображение данных массива
            case 2: {
                if (movies.empty()) {
                    cout << ERROR_MESSAGE_EMPTY_LIST << endl;
                } else {
                    cout << "\n=== ALL MOVIES ===" << endl;
                    for (int i = 0; i < movies.size(); ++i) {
                        cout << movies[i] << endl;
                    }
                }
                break;
            }
            /// Запись данных в файл
            case 3: {
                Movie::write_to_file(output_file_name, movies);
                break;
            }
            /// Выборка а) по режиссёру
            case 4: {
                Movie::select_by_director(movies, movies.size());
                break;
            }
            /// Выборка б) по жанру и рейтингу
            case 5: {
                Movie::select_by_genre_and_rating(movies, movies.size());
                break;
            }
            /// Выборка в) по году выхода
            case 6: {
```

```
            /// Выборка в) по году выхода
            case 6: {
                Movie::select_by_years(movies, movies.size());
                break;
            }
            /// Сортировка по рейтингу (по убыванию)
            case 7: {
                if (!movies.empty())
                    Movie::sort_by_rating(movies);
                else
                    cout << ERROR_MESSAGE_EMPTY_LIST << endl;
                break;
            }
            /// Добавление фильма в массив
            case 8: {
                Movie new_movie;
                cin >> new_movie;
                movies.push_back(new_movie);
                cout << "Movies added successfully." << endl;
                break;
            }
            /// Выход из программы
            case 9: {
                cout << "Exiting the program..." << endl;
                running = false;
                break;
            }
            /// Неверный выбор опции меню
            default:
                cout << ERROR_MESSAGE_INVALID_MENU_CHOICE << endl;
        }
    }

    return 0;
}
```

Загрузка данных из файла **input.txt** и вывод в консоль:

```
=====================================
                MAIN MENU
=====================================
1. Load data from file
2. Display data on screen
3. Save data to file
4. Select movies by director
5. Select movies by genre and rating
6. Select movies by years
7. Sort movies by rating (qsort)
8. Add new movie
9. Exit

Choose an action: 1
Data loaded successfully. Number of movies: 4

=====================================
                MAIN MENU
=====================================
1. Load data from file
2. Display data on screen
3. Save data to file
4. Select movies by director
5. Select movies by genre and rating
6. Select movies by years
7. Sort movies by rating (qsort)
8. Add new movie
9. Exit

Choose an action: 2

=== ALL MOVIES ===
Name: "Bad_movie"
Director: "Mr_Bad_Director"
Year: "2010"
Genre: "Thriller"
Rating: "3.7"
Length: "85"

Name: "Excellent_movie"
Director: "Mr_Talanted_Director"
Year: "1992"
Genre: "Western"
Rating: "9.2"
Length: "140"

Name: "Another_Thriller"
Director: "Mr_Bad_Director"
Year: "2015"
Genre: "Thriller"
Rating: "6.5"
Length: "120"

Name: "Some_Horror"
Director: "Mr_Lame_Director"
Year: "1997"
Genre: "Horror"
Rating: "4.8"
Length: "75"
```

Выборка фильмов по имени режиссёра:

```
=====================================
                MAIN MENU
=====================================
1. Load data from file
2. Display data on screen
3. Save data to file
4. Select movies by director
5. Select movies by genre and rating
6. Select movies by years
7. Sort movies by rating (qsort)
8. Add new movie
9. Exit

Choose an action: 4
Enter director name: Mr_Bad_Director

=== MOVIES BY DIRECTOR "Mr_Bad_Director" ===
Name: "Bad_movie"
Director: "Mr_Bad_Director"
Year: "2010"
Genre: "Thriller"
Rating: "3.7"
Length: "85"

Name: "Another_Thriller"
Director: "Mr_Bad_Director"
Year: "2015"
Genre: "Thriller"
Rating: "6.5"
Length: "120"
```

Выборка фильмов по жанру и рейтингу:

```
=====================================
                MAIN MENU
=====================================
1. Load data from file
2. Display data on screen
3. Save data to file
4. Select movies by director
5. Select movies by genre and rating
6. Select movies by years
7. Sort movies by rating (qsort)
8. Add new movie
9. Exit

Choose an action: 5
Enter genre: Thriller
Enter minimum rating: 5.0

=== MOVIES OF GENRE "Thriller" WITH RATING > 5 ===
Name: "Another_Thriller"
Director: "Mr_Bad_Director"
Year: "2015"
Genre: "Thriller"
Rating: "6.5"
Length: "120"
```

Выборка фильмов по году выхода:

```
=====================================
                MAIN MENU
=====================================
1. Load data from file
2. Display data on screen
3. Save data to file
4. Select movies by director
5. Select movies by genre and rating
6. Select movies by years
7. Sort movies by rating (qsort)
8. Add new movie
9. Exit

Choose an action: 6
Enter start year: 2000
Enter end year: 2025

=== MOVIES BETWEEN 2000 AND 2025 ===
Name: "Bad_movie"
Director: "Mr_Bad_Director"
Year: "2010"
Genre: "Thriller"
Rating: "3.7"
Length: "85"

Name: "Another_Thriller"
Director: "Mr_Bad_Director"
Year: "2015"
Genre: "Thriller"
Rating: "6.5"
Length: "120"
```

Сортировка фильмов по рейтингу (по убыванию):

```
=====================================
                MAIN MENU
=====================================
1. Load data from file
2. Display data on screen
3. Save data to file
4. Select movies by director
5. Select movies by genre and rating
6. Select movies by years
7. Sort movies by rating (qsort)
8. Add new movie
9. Exit

Choose an action: 7
Movies sorted by rating (descending).

=====================================
                MAIN MENU
=====================================
1. Load data from file
2. Display data on screen
3. Save data to file
4. Select movies by director
5. Select movies by genre and rating
6. Select movies by years
7. Sort movies by rating (qsort)
8. Add new movie
9. Exit

Choose an action: 2

=== ALL MOVIES ===
Name: "Excellent_movie"
Director: "Mr_Talanted_Director"
Year: "1992"
Genre: "Western"
Rating: "9.2"
Length: "140"

Name: "Another_Thriller"
Director: "Mr_Bad_Director"
Year: "2015"
Genre: "Thriller"
Rating: "6.5"
Length: "120"

Name: "Some_Horror"
Director: "Mr_Lame_Director"
Year: "1997"
Genre: "Horror"
Rating: "4.8"
Length: "75"

Name: "Bad_movie"
Director: "Mr_Bad_Director"
Year: "2010"
Genre: "Thriller"
Rating: "3.7"
Length: "85"
```

Добавление нового фильма в массив:

```
=====================================
                MAIN MENU
=====================================
1. Load data from file
2. Display data on screen
3. Save data to file
4. Select movies by director
5. Select movies by genre and rating
6. Select movies by years
7. Sort movies by rating (qsort)
8. Add new movie
9. Exit

Choose an action: 8
Enter name: NAME
Enter director: DIRECTOR
Enter year: 2025
Enter genre: GENRE
Enter rating: 10.0
Enter length: 120
Movies added successfully.

=====================================
                MAIN MENU
=====================================
1. Load data from file
2. Display data on screen
3. Save data to file
4. Select movies by director
5. Select movies by genre and rating
6. Select movies by years
7. Sort movies by rating (qsort)
8. Add new movie
9. Exit

Choose an action: 2

=== ALL MOVIES ===
Name: "Excellent_movie"
Director: "Mr_Talanted_Director"
Year: "1992"
Genre: "Western"
Rating: "9.2"
Length: "140"

Name: "Another_Thriller"
Director: "Mr_Bad_Director"
Year: "2015"
Genre: "Thriller"
Rating: "6.5"
Length: "120"

Name: "Some_Horror"
Director: "Mr_Lame_Director"
Year: "1997"
Genre: "Horror"
Rating: "4.8"
Length: "75"

Name: "Bad_movie"
Director: "Mr_Bad_Director"
Year: "2010"
Genre: "Thriller"
Rating: "3.7"
Length: "85"

Name: "NAME"
Director: "DIRECTOR"
Year: "2025"
Genre: "GENRE"
Rating: "10"
Length: "120"
```

Сохранение данных в файл **output.txt**:

```
=====================================
                MAIN MENU
=====================================
1. Load data from file
2. Display data on screen
3. Save data to file
4. Select movies by director
5. Select movies by genre and rating
6. Select movies by years
7. Sort movies by rating (qsort)
8. Add new movie
9. Exit

Choose an action: 3
Data saved to output.txt
```

```
You, 3 минуты назад | 1 author (You)
Name: "Excellent_movie"
Director: "Mr_Talanted_Director"
Year: "1992"
Genre: "Western"
Rating: "9.2"
Length: "140"

Name: "Another_Thriller"
Director: "Mr_Bad_Director"
Year: "2015"
Genre: "Thriller"
Rating: "6.5"
Length: "120"

Name: "Some_Horror"
Director: "Mr_Lame_Director"
Year: "1997"
Genre: "Horror"
Rating: "4.8"
Length: "75"

Name: "Bad_movie"
Director: "Mr_Bad_Director"
Year: "2010"
Genre: "Thriller"
Rating: "3.7"
Length: "85"

Name: "NAME"
Director: "DIRECTOR"
Year: "2025"
Genre: "GENRE"
Rating: "10"
Length: "120"
```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. **Раскройте понятие указателя в языке С. Опишите синтаксис объявления указателя.**
   Указатель — это переменная, которая хранит адрес другой переменной в памяти и позволяет к ней обращаться.
   Синтаксис объявления: *тип_данных *имя_указателя*;
2. **В чем преимущество использования операций new и delete в языке C++ по сравнению с аналогичными средствами языка C?**
   Операторы new и delete в C++ автоматически вызывают конструкторы и деструкторы объектов, обеспечивая правильную инициализацию и очистку. В отличие от malloc и free в C, new также является типобезопасным, возвращая указатель нужного типа без явного приведения.
3. **В чем заключаются преимущества использования динамических массивов?**
   Основные преимущества динамических массивов — это возможность изменять их размер во время выполнения программы, выделять память под массивы произвольного размера и управлять временем их жизни, которое не ограничено областью видимости.
4. **Как производится обращение к элементам динамических массивов?**
   Обращение к элементам динамических массивов производится так же, как и к элементам статических массивов — с помощью оператора индексации *arr[0] = 5* или через арифметику указателей **(arr + i)*.
5. **Как производится определение пользовательских функций в языке С?**
   Определение пользовательской функции в С включает в себя указание типа возвращаемого значения, имени функции, списка параметров в скобках и тела функции в фигурных скобках, содержащего исполняемый код.
6. **Как происходит передача параметров в функцию по значению/ адресу?**
   - **По значению:** в функцию передается копия значения переменной. Изменения внутри функции не влияют на оригинал.
   - **По адресу:** в функцию передается адрес переменной. Функция может изменить оригинал.
7. **Какого типа значения может возвращать функция? Как вернуть из функции несколько значений?**
   Функция может возвращать значение любого базового или производного типа данных или не возвращать ничего (тип *void*). Чтобы вернуть несколько значений, можно использовать структуру, передать параметры по ссылке или через указатель или использовать *std::tuple* и *std::pair.*
8. **Как организуется доступ к файлам? Какие виды доступа вы знаете? Каким образом можно считать/записать данные из текстового файла?**
   Доступ к файлам организуется через потоки (*iostream*) или стандартную библиотеку (*stdio.h*). Основные виды доступа: чтение, запись, добавление в конец, перемещение в конец при открытии, а также двоичный режим. Для

чтения/записи данных из текстового файла используются функции форматированного ввода/вывода (например, fscanf/fprintf в C, операторы >>/<< в C++) или функции для работы со строками (например, fgets/fputs).

9. **Что такое класс? Как выполнить объявление класса?**
   Класс — это пользовательский тип данных, который объединяет данные (атрибуты) и функции (методы), работающие с этими данными. Объявление класса выполняется с помощью ключевого слова *class*, за которым следует имя класса и тело класса в фигурных скобках, завершающееся точкой с запятой.
10. **Как в классе реализуется инкапсуляция?**
    Инкапсуляция — это сокрытие деталей реализации и предоставление интерфейса для работы с объектом. Это достигается с помощью модификаторов доступа (private, protected), которые ограничивают доступ к членам класса, предоставляя для взаимодействия с ними открытые методы (public).
11. **Каким образом элементы класса получают атрибуты доступа?**
    Элементы класса получают атрибуты доступа с помощью спецификаторов public, private и protected. Все члены, объявленные после спецификатора, имеют соответствующий уровень доступа до следующего спецификатора.
12. **Что такое объект класса и как выполнить объявление объекта класса в программе?**
    Объект — это конкретный экземпляр класса, созданный в памяти. Объявление: *имя_класса имя_объекта*.
13. **Что такое методы класса? Каковы особенности объявления и определения методов класса?**
    Методы класса — это функции, определенные внутри класса и работающие с его данными. Могут быть объявлены внутри класса (встроенные) или вне (с указанием имени класса через ::).
14. **Что такое указатель this? Как выполняется вызов метода класса?**
    this — это указатель, который хранит адрес текущего объекта, для которого вызван метод. Автоматически передается компилятором. Вызов метода класса выполняется с помощью специального оператора . или -> (для указателей на объекты).
15. **Какие функции называются конструкторами? Каков механизм передачи параметров конструктору?**
    Конструкторы — это специальные методы класса, которые автоматически вызываются при создании объекта. Они имеют то же имя, что и класс, и не имеют возвращаемого типа. Параметры передаются конструктору в скобках при объявлении объекта.
16. **Что такое конструктор по умолчанию и какую пользу приносит его наличие?**
    Конструктор — это специальная функция класса, которая вызывается автоматически при создании объекта. Используется для инициализации объекта. Может быть перегружен для определения конструкторов с разными

параметрами. Если ни один конструктор не определен, компилятор создает его автоматически.

17. **Как выполняется инициализация объектов класса?**
Инициализация объектов класса происходит при их создании с помощью конструкторов. Значения для членов класса могут передаваться как параметры конструктору, который затем присваивает их соответствующим полям. Также инициализация может происходить в списке инициализации.

18. **Каким образом в программе можно создать объект класса, элементы-данные которого являются копиями элементов данных созданного ранее объекта?**
Это можно сделать с помощью конструктора копирования. Он вызывается автоматически при инициализации нового объекта существующим, например: MyClass newObject = oldObject; или MyClass newObject(oldObject);.

19. **Каким образом осуществляется перегрузка операций в языке C++?**
Перегрузка операций — это возможность определить поведение операторов для пользовательских типов. Это делается через метод класса или дружественную функцию.

20. **Какие операции не могут быть перегружены?**
    - :: (область видимости)
    - .* (выбор через указатель на член)
    - . (выбор члена)
    - ?: (условный оператор)
    - sizeof
    - typeid
    - alignof

21. **Теряет ли операция при перегрузке свое исходное назначение?**
Нет. Перегрузка оператора не изменяет его поведение для встроенных типов, она работает только для пользовательских.

22. **Каков синтаксис объявления оператора-функции?**
return_type operator@ (parameters)

23. **Почему при перегрузке унарных операций оператор-функция не имеет параметров?**
Унарный оператор (например, -, !, ++) применяется к одному операнду — текущему объекту. Поэтому параметр не нужен — используется *this*.

24. **Как различить префиксную и постфиксную операции инкремента при перегрузке?**
Для различения используется фиктивный int параметр. Префиксная форма (++obj) объявляется без параметров (operator++()), а постфиксная (obj++) — с одним параметром типа int (operator++(int)), который не используется в теле функции.

25. **Какая функция называется другом класса?**
Дружественная функция — это функция, не являющаяся членом класса, но имеющая доступ к его private и protected членам. Она объявляется внутри класса с ключевым словом *friend*.

26. **В чем состоят различия между дружественной функцией и методом класса?**
    - Метод класса вызывается для конкретного объекта и получает на него указатель *this*.
    - Дружественная функция не привязана к объекту и не имеет указателя *this*. Все объекты, к которым ей нужен доступ, должны быть переданы ей в качестве явных параметров.
27. **В чем состоят особенности передачи параметров дружественной функции при перегрузке унарных и бинарных операций?**
    - Для унарной операции дружественная функция принимает один параметр — объект класса.
    - Для бинарной операции — два параметра (левый и правый операнды).
28. **Почему нельзя перегрузить операции >> и << для ввода и вывода типов с помощью метода класса?**

    Потому что левый операнд этих операторов — это объект потока (*istream* или *ostream*), а не объект класса. Метод класса требует, чтобы первым операндом был объект этого класса. Поэтому эти операторы должны быть дружественными функциями.

ВЫВОДЫ

Я повторил основы работы с классами в языке C++, проверил навыки использования языковых средств и способов динамического выделения памяти; организации форматного ввода и вывода. Получил практические навыки разработки и отладки программ, использующих объекты класса и файловый ввод/вывод. Изучил различные реализации конструкторов и области их применения, а также аспекты перегрузки операций и работы с дружественными функциями.